Cuestionario: Fundamentos y Configuración de Apache

Módulo: Despliegue de Aplicaciones Web (0614) - 2º DAW
Alumno/a: Javier Jiménez Alonso
Fecha: 10/11/2026

Lee atentamente los apuntes de la Unidad 2 (páginas 1 a 21) y responde de forma clara y justificada a las siguientes cuestiones técnicas.

1. Rutas y estructura predeterminada: Según el documento de la unidad, al instalar Apache en un sistema GNU/Linux Debian, ¿en qué directorio exacto se crea por defecto la página web inicial (index.html) y qué archivo de configuración contiene las directivas predeterminadas para dicha página?

Al instalar Apache en un GNU/Linux Debian se crea la página web inicial en `/var/www/index.html`. Esa página está referenciada desde el archivo `/etc/apache2/sites-available/default`, que es el que contiene la configuración por defecto generada en la instalación para esa página: el `DocumentRoot` y las directivas con las que Apache sirve el sitio.

El procedimiento que describen los apuntes es sustituir ese `index.html` por la página propia (o subir más páginas al mismo directorio) manteniendo siempre el nombre `index.html` para la página principal, porque es el nombre que Apache busca por defecto cuando se solicita el directorio.

Matiz de versión: el documento trabaja con Debian 6. En las versiones actuales de Debian y Ubuntu el `DocumentRoot` por defecto es `/var/www/html` y el archivo de configuración del sitio por defecto es `/etc/apache2/sites-available/000-default.conf`; el funcionamiento es el mismo, solo cambian la ruta y el nombre del fichero.

2. Concepto técnico de página estática: Basándote en el texto, justifica por qué una página web cuyo contenido es actualizado manualmente con cierta frecuencia (por ejemplo, un manual de instrucciones) sigue considerándose "estática" y qué requisito técnico marca la diferencia con una página "dinámica".

Un manual de instrucciones que se actualiza a mano sigue siendo una página **estática** porque lo que define el concepto no es la frecuencia con la que cambia el contenido, sino **quién genera ese contenido**. En una página estática el servidor se limita a entregar el archivo tal y como está guardado en disco: no se

ejecuta código en el servidor (por ejemplo PHP) ni en el cliente (por ejemplo JavaScript) para construir la respuesta. Que el administrador edite el HTML y lo vuelva a subir no cambia nada de eso; el servidor sigue sirviendo un fichero ya escrito.

El requisito técnico que marca la diferencia es, por tanto, la **necesidad de ejecutar código** al atender la petición: si para mostrar la página hace falta que se ejecute código en el servidor y/o en el cliente, o que se consulte una base de datos, la página es **dinámica**; si basta con entregar el archivo HTML/XHTML junto con su CSS, es estática.

3. Impacto del contenido dinámico: ¿Qué consecuencias tiene para un servidor web tener que ofrecer páginas dinámicas en lugar de estáticas en términos de consumo de recursos (CPU y memoria), administración y seguridad (gestión de módulos)?

Ofrecer páginas dinámicas tiene tres consecuencias para el servidor:

- **Consumo de recursos:** cada petición obliga a ejecutar código (PHP, CGI, Perl, etc.), lo que consume más **tiempo de CPU** y más **memoria** que servir un fichero estático. El resultado se tiene que construir en cada petición, y eso se nota en el rendimiento y en el número de usuarios simultáneos que puede atender.
- **Administración:** la configuración es más compleja. Hay que instalar, configurar y mantener los módulos que dan soporte a ese contenido (`mod_php`, `mod_cgi`, `mod_perl`, `mod_python`, `mod_include`, `mod_ldap`...). Cuantos más módulos, más trabajo de configuración y de actualización.
- **Seguridad:** cada módulo añadido es una superficie de ataque más. Cuantos más módulos haya cargados, más probabilidades de sufrir un problema de seguridad, y además hay que controlar qué código se ejecuta en el servidor, porque ese código corre con los permisos del usuario del servicio web.

4. Comunicaciones HTTP vs HTTPS: Explica las diferencias fundamentales en el flujo de información entre el navegador y el servidor al utilizar el protocolo HTTP frente a HTTPS, indicando además qué puertos TCP tiene asignados por defecto cada uno en la configuración de Apache.

HTTP envía la información en **texto claro**. El flujo es: el navegador traduce el dominio a una IP mediante DNS, conecta con el servidor en el **puerto TCP 80** (el que Apache escucha por defecto para HTTP, definido con la directiva `Listen`) y se intercambian la petición y la respuesta sin cifrar. Cualquiera que capture el tráfico de la red puede leer el contenido e incluso modificarlo.

HTTPS añade una capa de cifrado con **SSL/TLS** sobre el mismo esquema. El flujo es parecido, pero usa el **puerto TCP 443** por defecto y, antes de transferir la página, se abre una **negociación SSL**: el servidor presenta su certificado, el

navegador comprueba que lo ha firmado una entidad de certificación en la que confía y, si es válido, se acuerda el cifrado de la conexión. A partir de ahí la información viaja cifrada y con garantías de autenticidad e integridad.

El precio del cifrado es cómputo: HTTPS consume más CPU en el servidor que HTTP. Por eso a veces se protege solo la autenticación o determinadas zonas del sitio. Los dos protocolos pueden convivir en el mismo dominio.

5. Certificados y confianza: El servidor Apache tiene la capacidad de emitir sus propios certificados digitales. Si utilizamos un certificado creado por el propio servidor para una página accesible de forma pública en Internet, ¿cómo reaccionará el navegador del cliente al recibirlo y cuál es el motivo técnico de esta reacción?

Si el certificado lo ha emitido el propio servidor (autofirmado) y la página es pública en Internet, el navegador mostrará un **aviso de seguridad**: conexión no privada, certificado no válido o riesgo de suplantación, y pedirá al usuario que acepte la excepción manualmente para continuar.

El motivo técnico es que los navegadores llevan una lista de **entidades de certificación (CA) de confianza** y solo aceptan certificados cuya cadena de firma termina en una de ellas. En un certificado autofirmado el emisor y el titular son el mismo servidor, así que no hay una tercera parte de confianza que lo avale y el navegador no puede validar la cadena. No significa que el cifrado sea malo, pero sí que no hay forma de comprobar que ese certificado pertenece realmente a ese dominio.

Por eso los certificados autofirmados se usan en entornos internos o de pruebas, donde el cliente se configura para confiar en ellos, y para un sitio público se contrata un certificado a una CA reconocida.

6. Estándar y cabeceras MIME: Define la utilidad del estándar MIME en las comunicaciones web. Indica mediante qué campo exacto de la cabecera de respuesta HTTP avisa el servidor al navegador del tipo de datos, y qué hace el programa cliente si un archivo carece de un tipo MIME especificado.

El estándar **MIME** (*Multipurpose Internet Mail Extensions*) especifica cómo un programa debe transferir archivos de texto, imagen, audio, vídeo o cualquier otro que no esté codificado en US-ASCII, y permite que el receptor sepa **qué tipo de archivo** está recibiendo para abrirlo con el programa adecuado. Se define en varios RFC (2045, 2046, 2047, 4288, 4289 y 2077).

El servidor avisa del tipo de datos mediante el campo `Content-Type` de la cabecera de respuesta HTTP. Por ejemplo, `Content-Type: text/html` indica que el contenido es una página HTML y `Content-Type: text/css` una hoja de estilos; el identificador tiene dos partes, la categoría general (`text` , `image` , `video` ...) y el tipo concreto (`html` , `gif` , `mp4` ...).

Si un archivo no tiene un tipo MIME especificado, el programa cliente puede **deducirlo por la extensión** del archivo: por ejemplo, un `.txt` lo interpreta como texto. También puede venir indicado en el propio código de la página (atributos `type` de `link`, `script`, `object`, `form`, `meta`).

7. Directivas MIME en Apache: Diferencia el comportamiento y la finalidad técnica de las directivas DefaultType y ForceType a la hora de configurar cómo debe servir Apache los archivos.

Las dos directivas sirven para decidir el `Content-Type` que envía Apache, pero actúan en momentos distintos:

- `DefaultType` es un **valor de respaldo**. Asigna el `Content-Type` indicado a cualquier archivo cuyo tipo MIME **no se pueda determinar por la extensión**. Es decir, primero Apache mira la extensión en `/etc/mime.types` (a través de `mods-available/mime.conf`) y, solo si no la reconoce, aplica el `DefaultType`. Por ejemplo, `DefaultType text/plain` hace que un archivo desconocido se muestre como texto.
- `ForceType` es una **imposición**. Hace que todos los archivos a los que se aplique esa directiva se sirvan **como el tipo MIME que se indique, aunque su extensión diga otra cosa**. Por ejemplo, `ForceType image/gif` sirve como imagen GIF los ficheros de ese ámbito, tengan la extensión que tengan.

En resumen: `DefaultType` solo entra cuando Apache no sabe qué tipo es el archivo; `ForceType` manda siempre y sustituye a lo que diga la extensión.

8. Concepto base de VirtualHosts: ¿Para qué sirve configurar hosts virtuales (virtualhosts) en un servidor web y cómo interactúan las directivas definidas de forma global en el archivo principal apache2.conf con cada uno de los virtualhosts creados de forma independiente?

Los **hosts virtuales** (`VirtualHost`) permiten que **un mismo servidor web aloje varios dominios o sitios distintos**, cada uno con su propio `DocumentRoot`, su `ServerName` y sus `ServerAlias`. Sin ellos, un servidor solo podría servir un sitio; con ellos, en la misma IP y el mismo Apache pueden convivir `empresa1.com`, `empresa2.com` ... y cada petición se atiende con la configuración del virtualhost cuyo nombre coincide con el dominio solicitado.

Cada virtualhost se define en un archivo independiente dentro de `/etc/apache2/sites-available/`, lo que permite activarlo o desactivarlo por separado con `a2ensite` y `a2dissite`.

La relación con la configuración principal es de **herencia**: todo lo que no esté definido dentro de un `VirtualHost` se hereda de `apache2.conf`. Por eso las directivas comunes a todos los sitios (por ejemplo `Timeout 300`) se escriben una

sola vez en la configuración principal, y no repetidas en cada virtualhost. Si una directiva aparece en el virtualhost, prevalece sobre la global para ese sitio.

Existen tres tipos: basados en **nombre** (varios dominios en la misma IP, el método habitual), basados en **IP** (una IP por sitio, más difícil de mantener) y basados en **varios servidores principales** (varios `apache2.conf` independientes, solo para casos complejos).